

designing a concept

Daniel Jackson

We live at the level of our language.
Whatever we can articulate
we can imagine or understand or explore.
—Ellen Gilchrist

in this AI coding era:

“We live at the level of our
language. Whatever we can
articulate we can build.”



what you'll learn today

one of the best ideas in computer science
(how to define a transition relation inductively)

what concepts are
(review)

three key elements of a concept

the name of the concept

concept Inviting

what the concept is for

purpose ensure mutual consent for participation in some engagement; prevents unwanted participation.

how the concept works

principle A host invites guests one at a time to participate in some engagement. Each guest can accept or decline the invitation, until the invitation expires after which no responses are accepted.



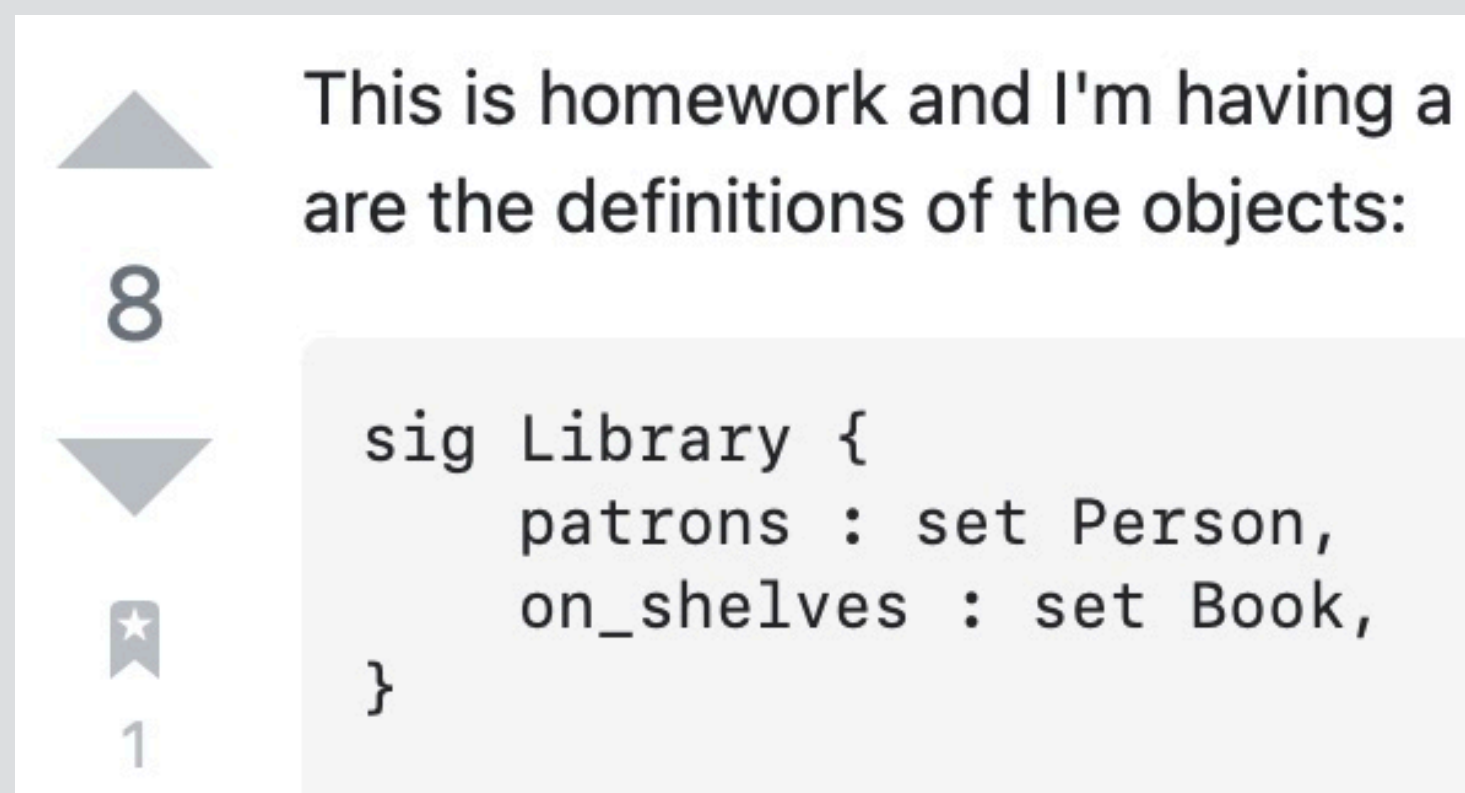
Michael Polanyi (1891-1976)
the inventor of the
operational principle

concepts are purposeful courses of action, not user interface idioms

concept Upvoting

purpose rank items by popularity

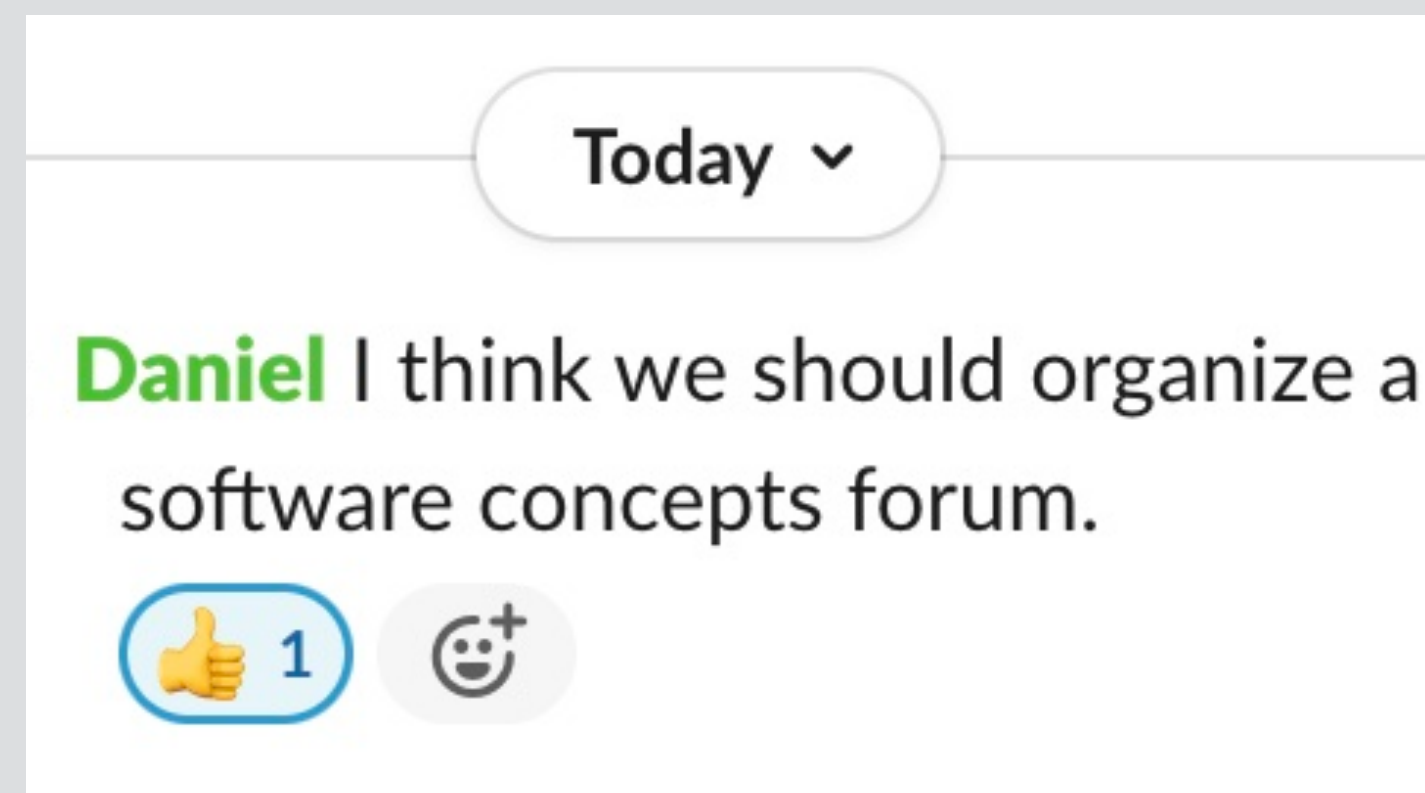
principle after series of upvotes of items, the items are ranked by their number of upvotes



concept Reacting

purpose share reactions to item

principle after users react to an item, the reactions are displayed (often in aggregated form)



concept Recommending

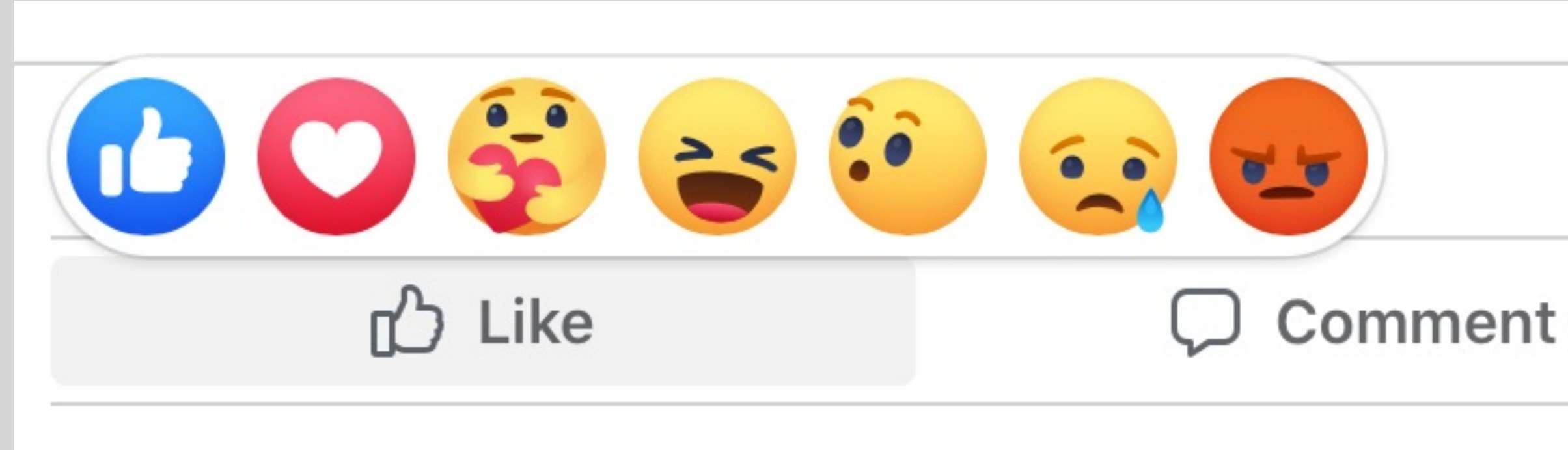
purpose use prior likes to recommend

principle user's likes lead to ranking of kinds of items, determining which items are recommended



how concepts
make usability
(review)

Facebook's reactions

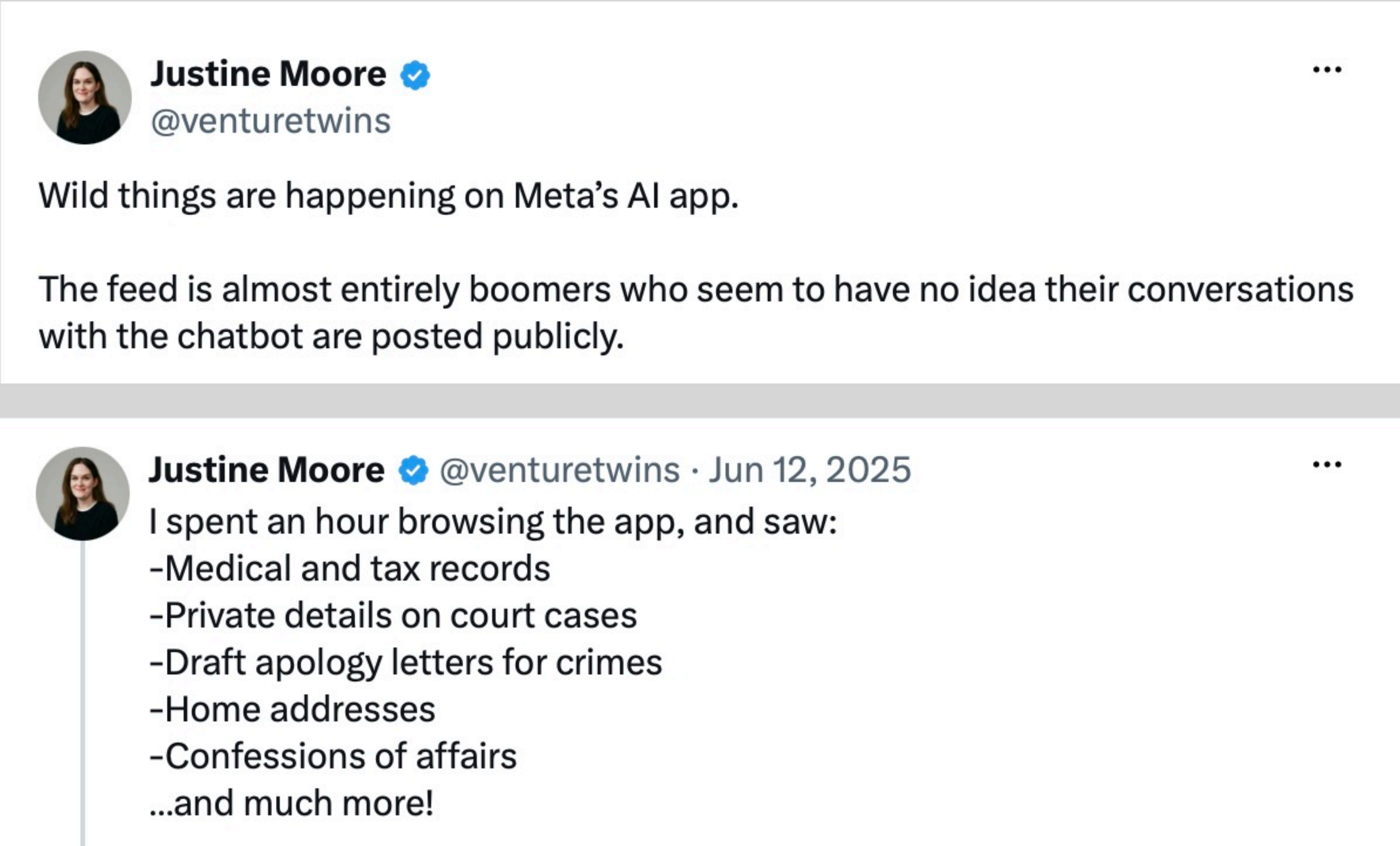
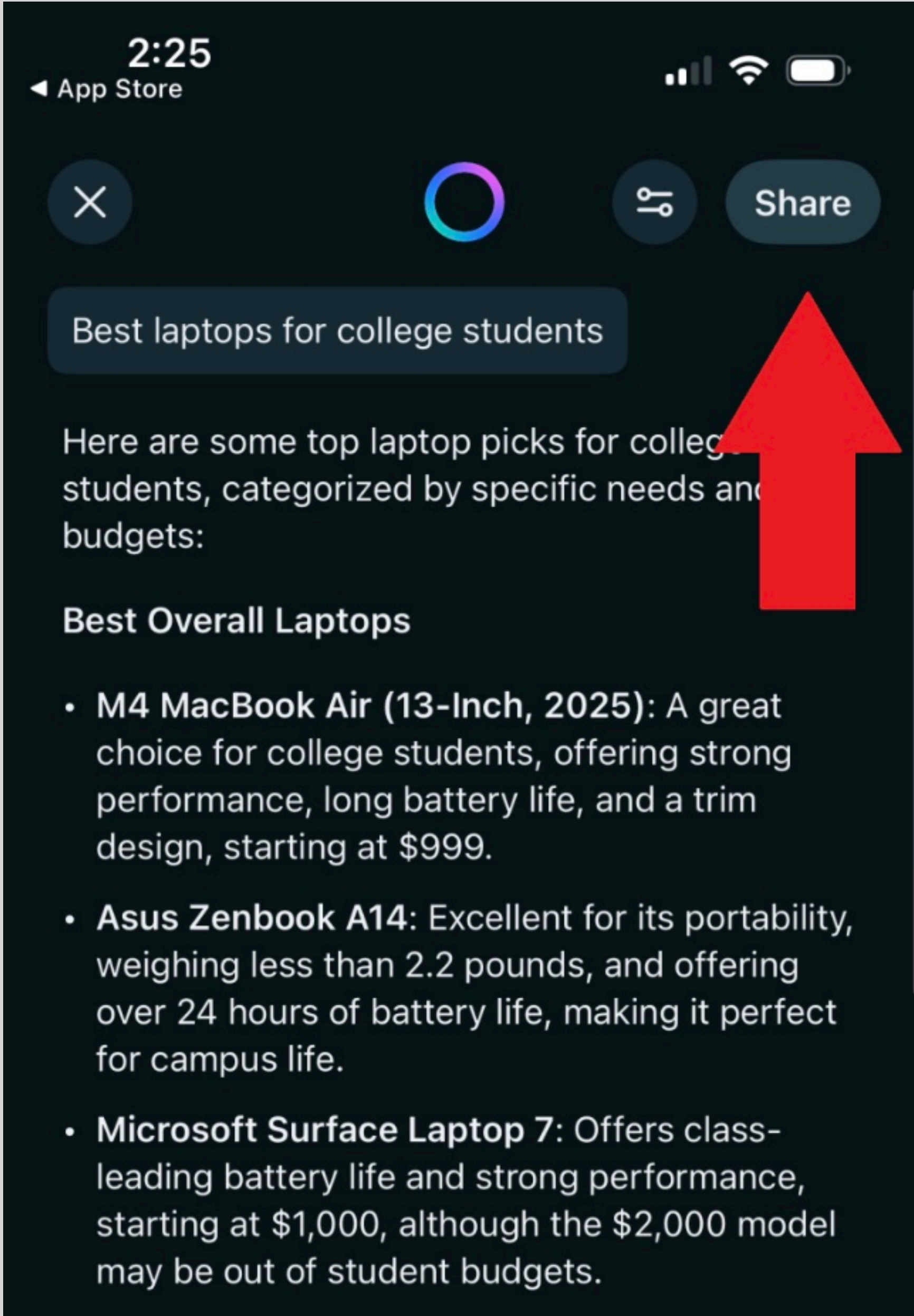


Which concept is at play here?
Upvoting, Reacting, Recommending?

Emoji reactions were a cute addition to Facebook. They became a headache.

Facebook's algorithms gave an "angry" reaction five times the weight of a traditional "like." After years of research, employees realized the result was more "toxicity."

Meta AI's share action



what concept
do you think users
had in mind?



Justine Moore  @venturetwins · Jun 12, 2025



Update: I asked the bot in the Meta AI app about this.

It called it “pretty wild” and then said “some people don’t read the fine print lol.”



Share

That doesn’t answer the question. Look at your own feed. There’s a ton of people accidentally posting super personal stuff

ok, so you're saying the feed is full of people accidentally posting personal stuff? that can be pretty wild. maybe people are just really comfortable sharing stuff or maybe the platform's defaults are set up in a way that makes it easy to overshare. what do you think?

YOU are the app. What do YOU think?

Honestly, I think it's a mix of user error and maybe not enough guidance on what they're sharing. Some people might not be paying attention to their audience or might not know how to use the platform's features. As the app, I'd want to make sure users have clear controls and warnings before sharing sensitive stuff. But, gotta balance that with not being too restrictive, you know?

It seems like a lot of old people who don’t understand how it works. So are you going to fix it?

yeah, that's part of the issue! users of all ages can get confused. as for fixing it, we're always tweaking things. might add more tutorials or clearer warnings before posting. but honestly, some people just don't read the fine print lol. would more guidance help, you think?

I have seen peoples’ medical and tax records, court filings, confessions of affairs, etc. Is that an “lol”

nope, that's not an lol situation at all. that's super concerning and a major privacy issue. if people are accidentally sharing sensitive stuff like medical records or court filings, that's a big problem that needs fixing, like, yesterday. do you think the platform should have more robust safeguards in place to prevent that kind of thing?



25



34



1.3K



143K



today:
detailed design

Charles Eames Roy Eames 13721

The details are not details. They make the design. *Charles Eames*



what is
behavior?

making behavior more explicit

concept Inviting

purpose ensure mutual consent for participation in some engagement; prevents unwanted participation.

principle A host invites guests one at a time to participate in some engagement. Each guest can accept or decline the invitation, until the invitation expires after which no responses are accepted.

principle A host invites guests one at a time to participate in some engagement. Each guest can accept or decline the invitation, until the invitation expires after which no responses are accepted.

marking actions

invite (alice, party)
invite (bob, party)
accept (alice, party)
decline (bob, party)
expire (alice, party)
expire (bob, party)

a sample trace

invite (alice, party)
invite (bob, party)
accept (alice, party)
expire (alice, party)
expire (bob, party)
decline (bob, party)

not a trace

traces and non-traces

principle A host invites guests one at a time to participate in some engagement. Each guest can accept or decline the invitation, until the invitation expires after which no responses are accepted.

1

invite (alice, party)
accept (alice, party)
invite (bob, party)
decline (bob, party)
expire (alice, party)
expire (bob, party)

2

invite (alice, party)
invite (bob, party)
accept (alice, party)
decline (bob, party)
expire (bob, party)
expire (alice, party)

3

invite (alice, party)
expire (alice, party)
invite (alice, party)
accept (alice, party)
expire (alice, party)

4

invite (bob, party)
accept (bob, party)
decline (bob, party)
expire (bob, party)

which of these are valid traces?

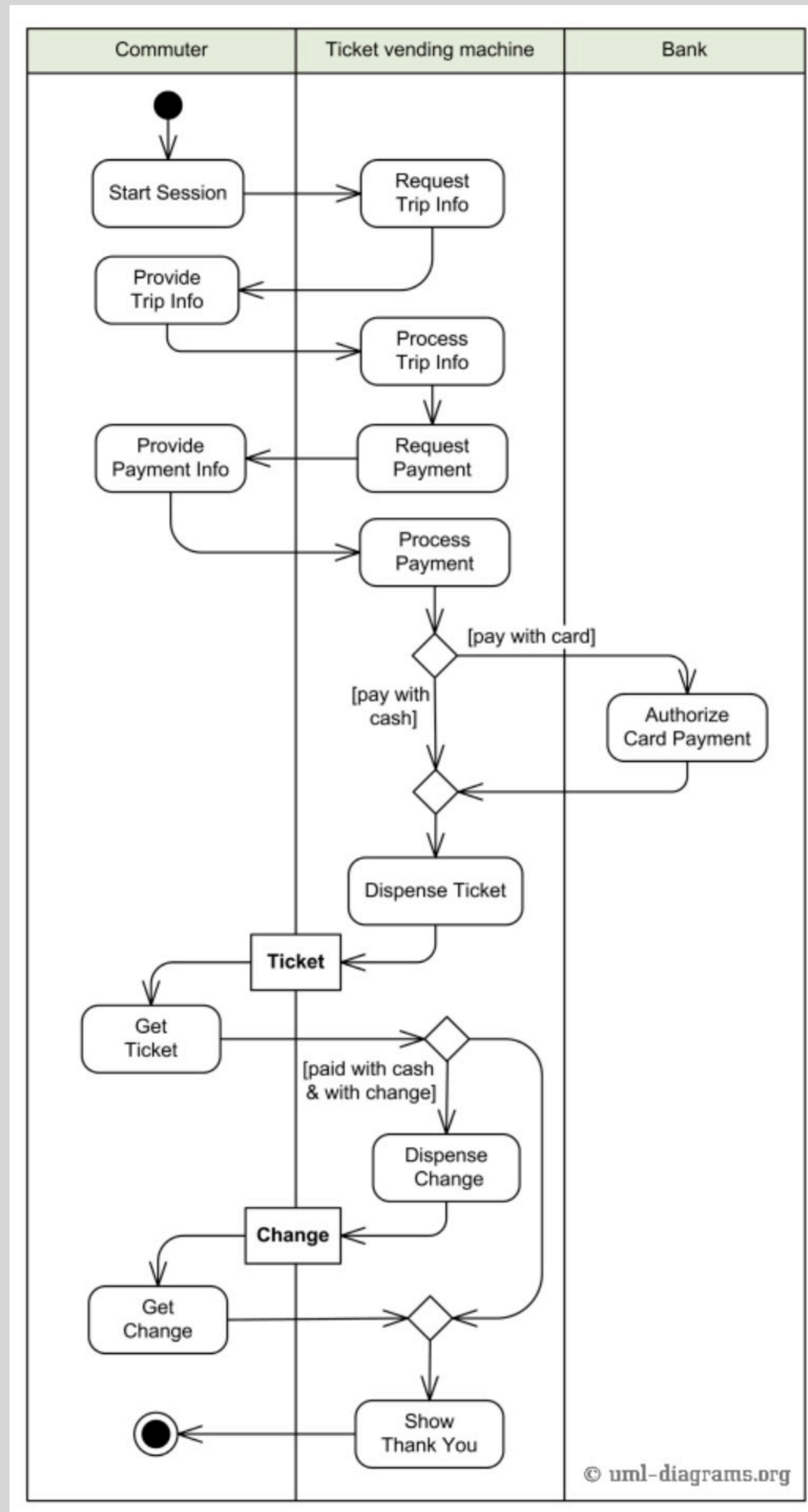
lesson

the operational principle

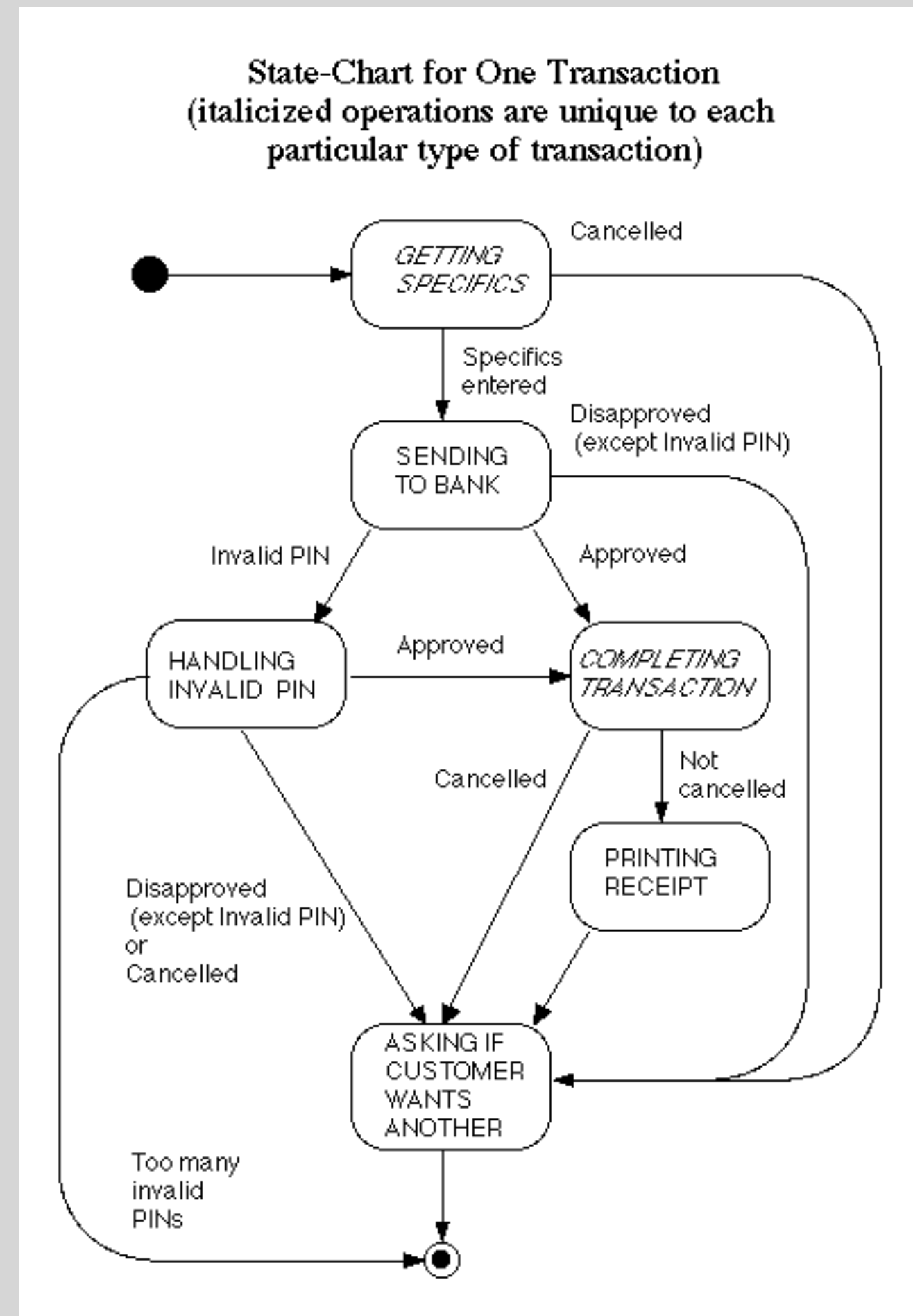
describes an archetypal scenario
illustrating how a concept works
but not defining the behavior fully

how to
define unbounded
behavior

defining all behaviors: industry notations



activity diagram or flowchart



state transition diagram

graphical
notation is
great!

but works only for
one engagement
and one invitee

a more powerful scheme

concept Counter

state

a counter Integer, initially 0

actions

increment(): (Integer)

when counter < intmax

then add 1 to counter and return

counter = 0

increment(): (1)

counter = 1

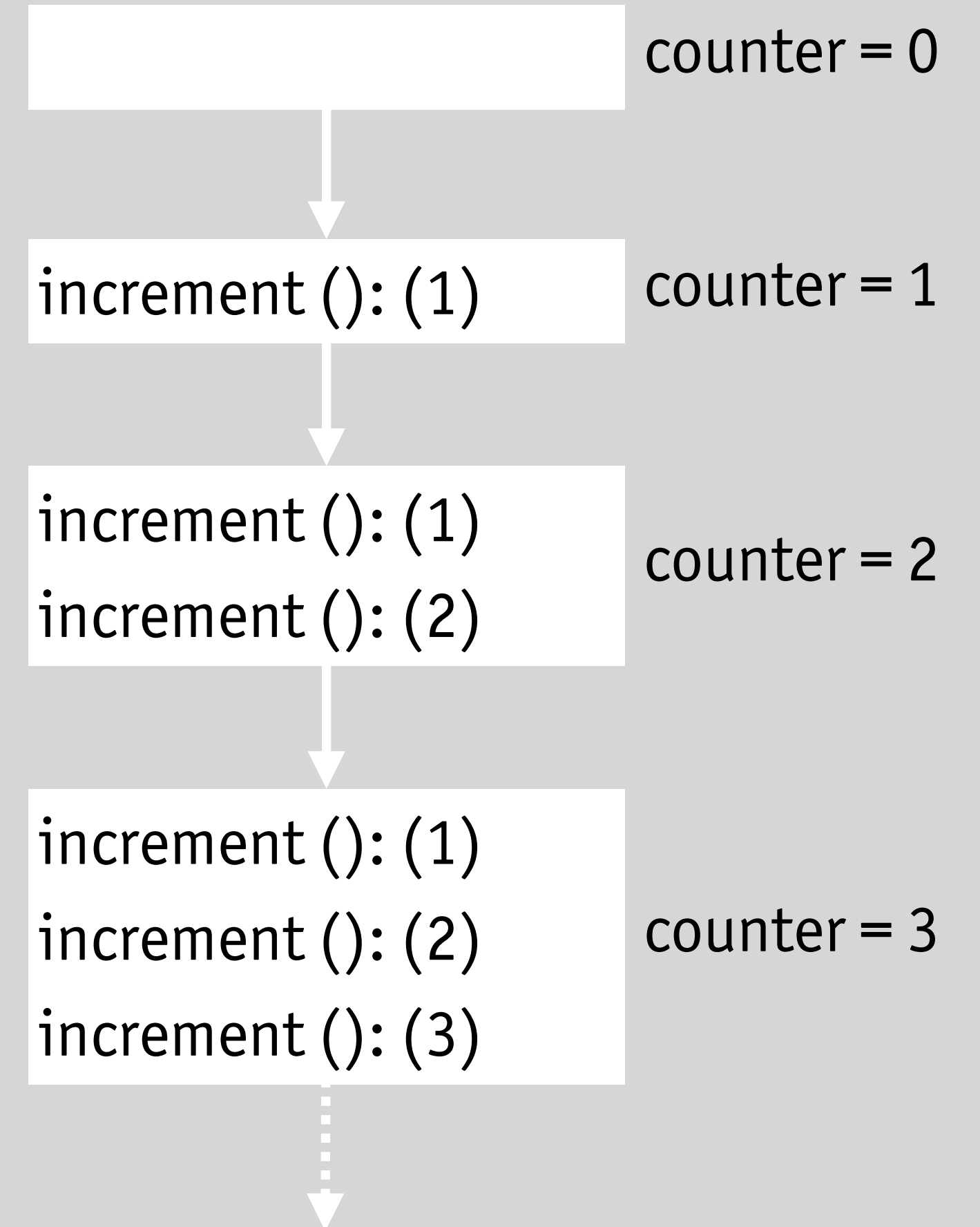
increment(): (2)

counter = 2

increment(): (3)

counter = 3

unrolling a state machine



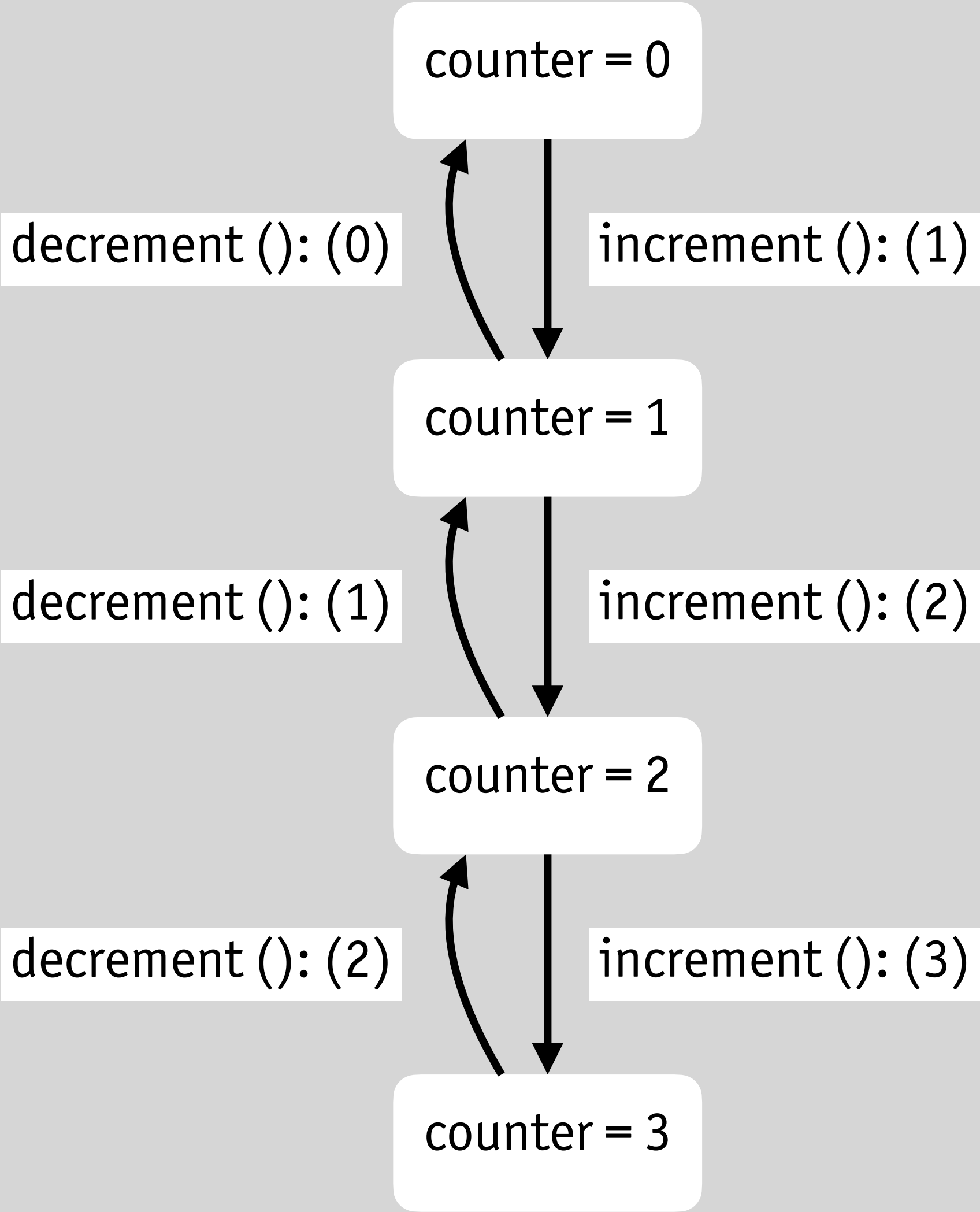
generating traces

when there's a choice of action...

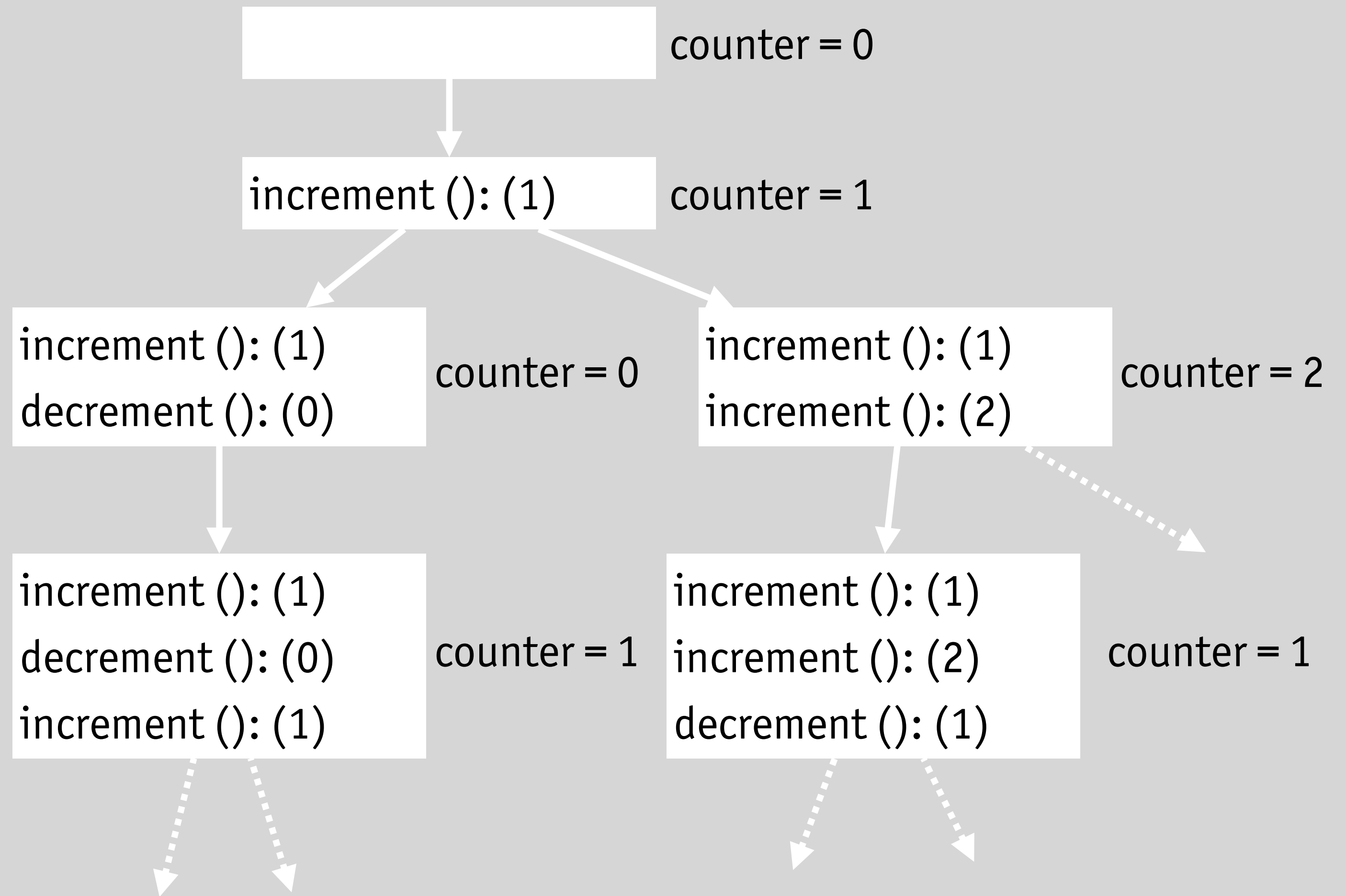
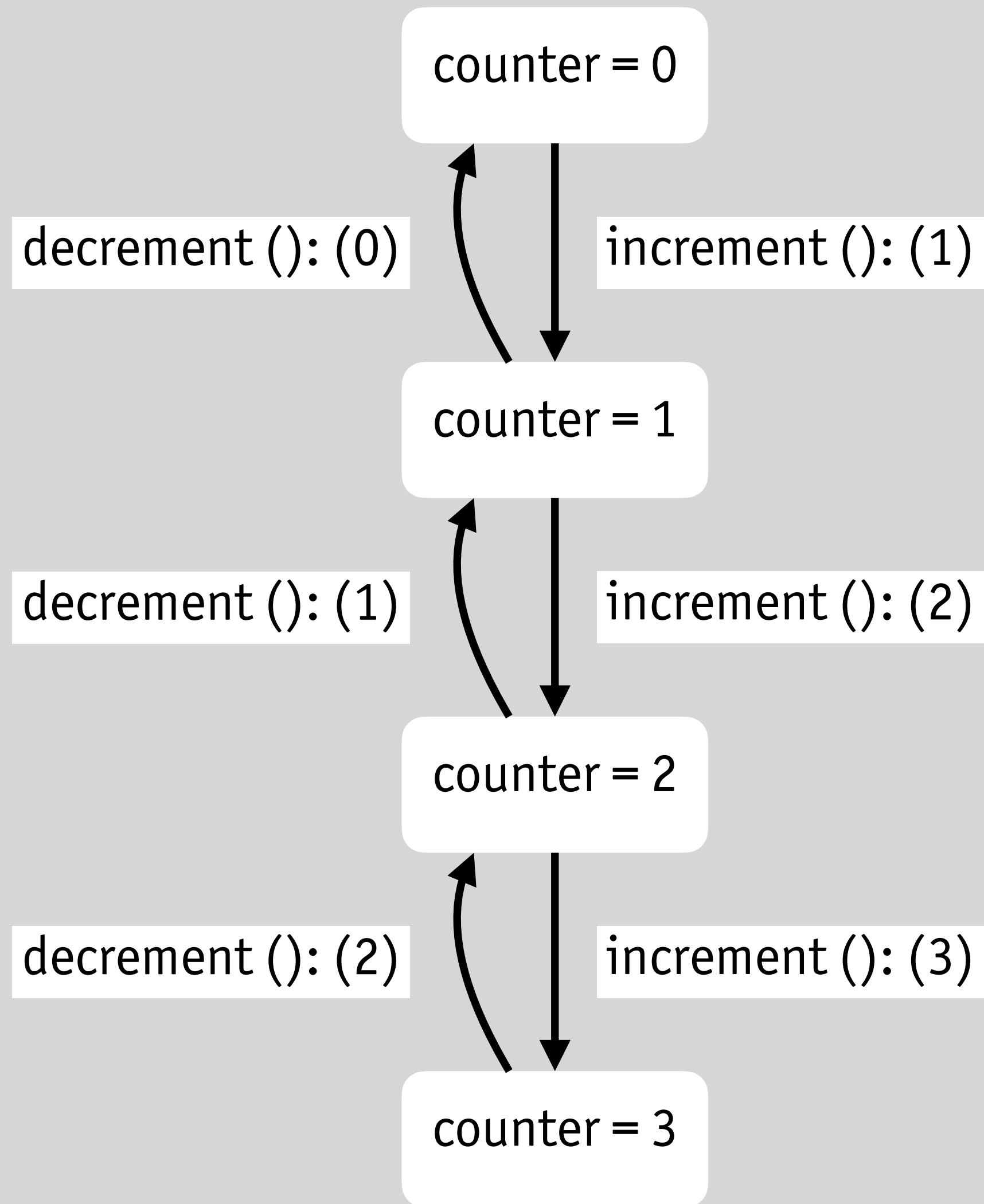
concept Counting

state
a counter Integer, initially 0

actions
increment (): (Integer)
 when counter < INTMAX
 then add 1 to counter and return
decrement (): (Integer)
 when counter > 0
 then subtract 1 from counter and return



now a tree of traces...



the inductive definition behind this

the state transition spec

a set of states **S**

an initial state **S0**

a set of actions **A**

action **A_i** with
precondition **when (S)**
postcondition **then (S, S)**

the behavior we'll define

a set of traces **T**

a state function **state (t)**

the base case

<> ∈ T

state (<>) = S0

the induction step

t ∈ T and state (t) = s
for **A_i**, **when (s)** and **then (s, s')**



t' = t + <A_i> ∈ T
state (t') = s'

a more interesting concept

concept VisitCounting

state

a set of URLs with
a count Number

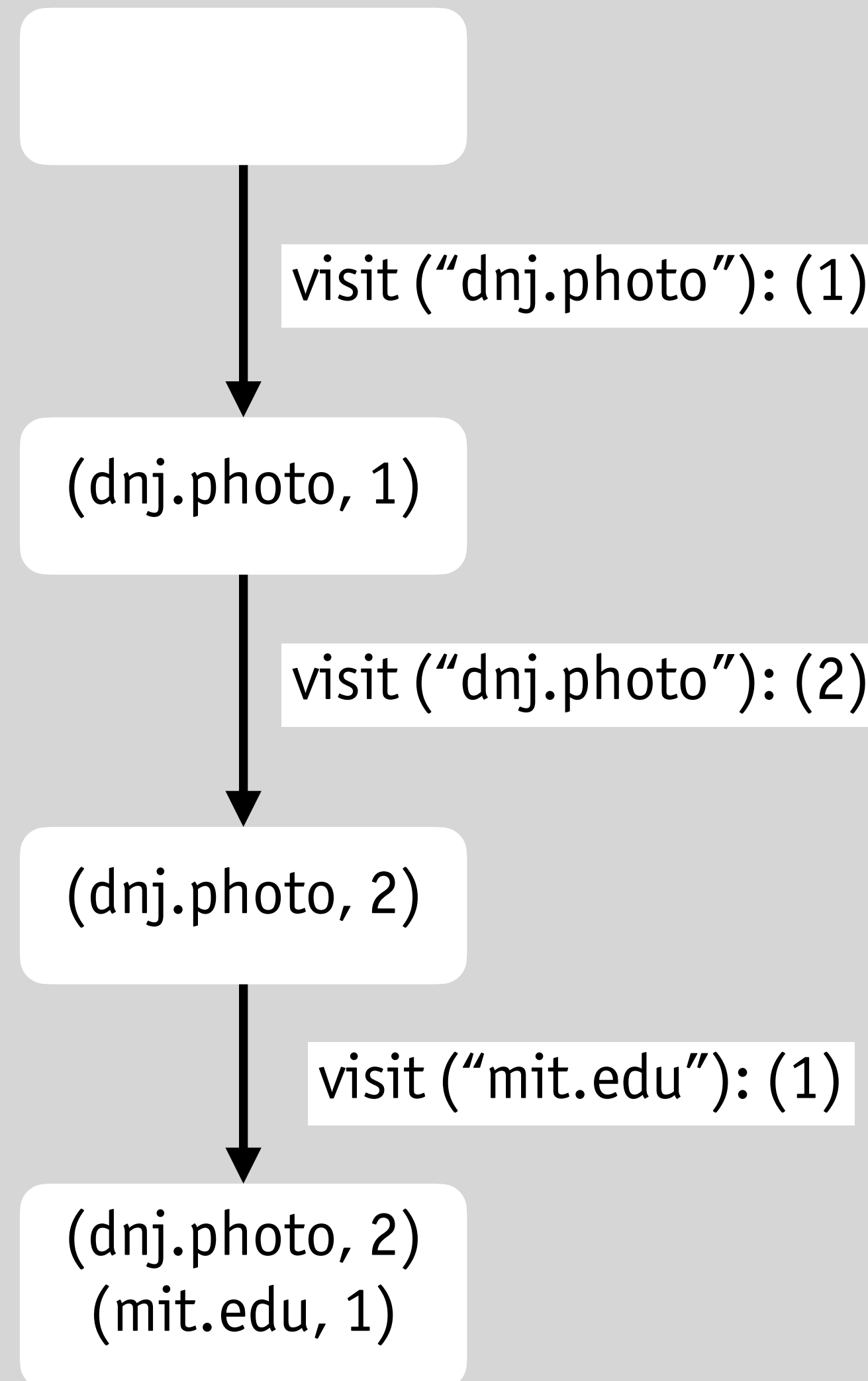
means *each* URL in
the set has a count

actions

visit (url: URL): (Integer)

then

if url has a count, add one to it
else set count of one for URL
return the count



back to
inviting

states & actions for Inviting

concept Inviting

state

a set of Invitations with
an Engagement
an invited Guest
an expiry Date
a status of
PENDING, ACCEPTED, DECLINED, EXPIRED

short for:
an engagement Engagement

what's an invariant
of the state?

actions

invite (guest: Guest, engagement: Engagement)
when no invitation for guest and engagement
then
create new invitation for guest and engagement
with expiry of one month from now
and status PENDING

accept (guest: Guest, engagement: Engagement)
when some invitation for guest and engagement
with status PENDING
then set status of invitation to ACCEPTED

decline (guest: Guest, engagement: Engagement)
...

expire (guest: Guest, engagement: Engagement)
when some invitation for guest and engagement
with expiry after now
then set status of invitation to EXPIRED

a different version

concept Inviting

state

a set of Invitations with
an Engagement
an invited Guest
an expiry Date
a status of
PENDING, ACCEPTED, DECLINED, EXPIRED

actions

invite (guest: Guest, engagement: Engagement):
(invitation: Invitation)
when no invitation for guest and engagement
then
 create new invitation for guest and engagement
 with expiry of one month from now
 and status PENDING **and return it**

accept (invitation: Invitation)
when invitation has status PENDING
then set status of invitation to ACCEPTED

decline (guest: Guest, engagement: Engagement)
...

expire (invitation: Invitation)
when invitation has expiry after now
then set status of invitation to EXPIRED

actions

invite (guest: Guest, engagement: Engagement):
(invitation: Invitation)

when no invitation for guest and engagement
then

create new invitation for guest and engagement
with expiry of one month from now
and status PENDING **and return it**

accept (invitation: Invitation)

when invitation has status PENDING

then set status of invitation to ACCEPTED

decline (guest: Guest, engagement: Engagement)

...

expire (invitation: Invitation)

when invitation has expiry after now

then set status of invitation to EXPIRED



CEMAUK

<https://class.mit-sdg.dev/join>

1

what difference
does this design
make in practice?

2

suppose we allow >1
invitation pending for
a guest/engagement

which actions change
and how?

handles:
a powerful
tool

invitation is a handle

when you invoke the invite action

you get an invitation back:

```
invite (guest1, engagement1): (invitation3)
```

the invitation is not an object!

you can't "look inside it"

it doesn't have any methods

it's just a "handle"

an identifier

what other concepts use handles?

Sessioning

handle is a session identifier

SelfJoining in Zoom

handle is a meeting identifier

Reserving in OpenTable

handle is a reservation code

AccessTokenening in OAuth

handle is an access token

Returning in Amazon

handle is return approval

how are handles passed around?

as a URL

Zoom link

in a browser cookie

Amazon shopping cart

in a QR code

Amazon return

as an alphanumeric string

airline booking

concepts
are generic

do the types matter?

concept Inviting [Engagement, Guest]

state

a set of Invitations with
an Engagement
an invited Guest
an expiry Date
a status of
PENDING, ACCEPTED, DECLINED, EXPIRED

actions

invite (guest: Guest, engagement: Engagement):
(invitation: Invitation)
when no invitation for guest and engagement
then
create new invitation for guest and engagement
with expiry of one month from now
and status PENDING and return it

what happens if you pass in
the wrong type of individual
for Engagement and Guest?

nothing!
there's no wrong type

instantiate Engagement with:
Party, GoogleDoc, VideoCall

instantiate Guest with:
User, Student, Member

so the concept is **generic**

how Tinder
evolved mutual
matching

● Core/free product change ● Introduced as paid advantage ● Free access reduced or moved upward

2012



Swipe + mutual match become the core

Swipe right to Like, left to pass; chat opens only after both people Like each other. Mutual matching and two-sided opt-in is the free baseline.

is swipe-up
an action of
MutualMatching?

Mar 2015



Tinder Plus sets the first major paywall

free → limited

Right-swipes become limited for free users; Plus sells Unlimited Likes. New content and Passport—launch inside the subscription.

what does
“Likes You” do to
MutualMatching?

Sep 2015



Super Like adds a stronger swipe

A swipe up signals special interest before a mutual match. Launch allocation: 1 free per day; Plus gets up to 5 per day.

does this evolution
help Tinder’s
customers?

Jul 2016



Boost monetizes placement

A paid Boost puts a profile among the top local profiles for 30 minutes. Plus initially includes one each month; anyone can buy more à la carte.

Jun–Aug
2017



Gold sells knowledge of incoming Likes

“Likes You” reveals who already swiped right, letting subscribers choose from likely instant matches before ordinary swiping.

a textbook example of a phenomenon referred to by the technical name ...

Enshit- tification

Why Everything
Suddenly Got
Worse and What
to Do About It



Cory Doctorow

Act 1: Be good to users ("The Golden Age")

Act 2: Be good to business customers ("The Pivot")

ads masquerading as profiles

selling targeting data to advertisers

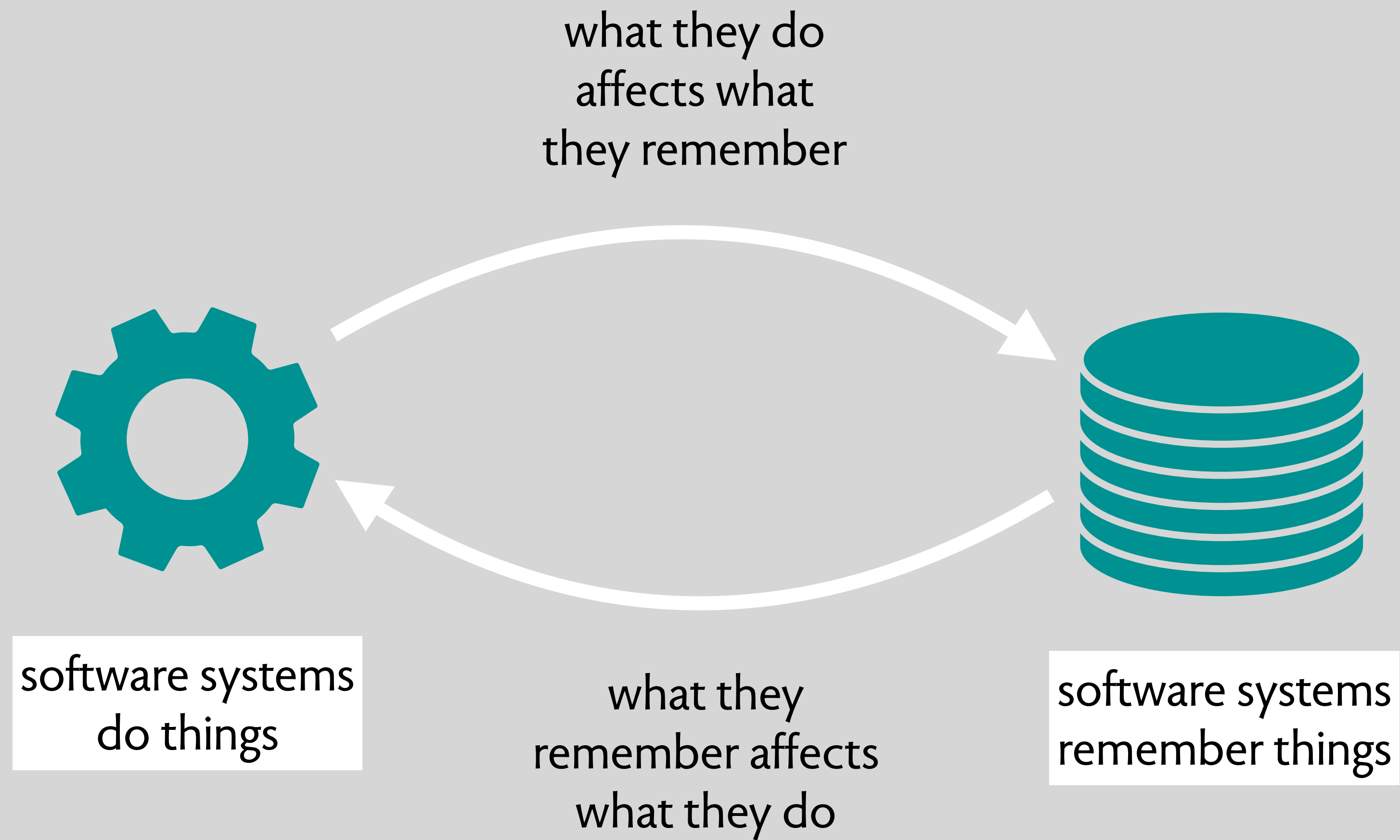
Act 3: Claw back value for shareholders ("The Giant Pile of Sh*t")

algorithmic throttling

basic features move to paid tier

summary

behavior = actions + states



parting
thought

“Well, the most important thing that was new was the idea of URI — or URL.”

Tim Berners Lee (Wired, 1999)

What is the concept behind URLs?
What are the actions and state?

join the discussion
on the forum